

# TLCTC Framework Glossary

Version 2.0 / 2.1

BK

Bernhard Kreinz

• Last updated: 24 Mar 2026 • Loading read time...

## A

### Abuse of Functions (#1)

A threat cluster where an attacker misuses the logic, scope, or configuration of existing, legitimate software functions for malicious purposes. This manipulation occurs through standard interfaces using expected input types (data, parameters, configurations, sequence of actions), but in a way that subverts the intended purpose or security controls. Crucially, inputs remain data; no foreign code is introduced or executed. The generic vulnerability is the scope, complexity, or inherent trust placed in legitimate software functions.

Classification is governed by the R-ABUSE mapping rule: if the attacker's success does not require any implementation flaw and instead abuses intended functionality, scope, or configuration via standard interfaces using expected input types, the step **MUST** be classified as **#1 Abuse of Functions**.

**Reference:** R-ABUSE (§2.2.4)

### Accessibility (Data Risk Event)

The operational state in which data or resources can be used for their intended purpose by authorized processes (Facility, IT, or Business processes). Loss of Accessibility occurs when data exists but cannot be used—such as encrypted files (ransomware), corrupted data, or permission lockouts. This is distinct from Loss of Availability: ransomware causes Loss of Accessibility (data present but unusable), not Loss of Availability (data gone or unreachable). DRE notation: **AC** when distinguished from Availability, or **A** as general shorthand covering both.

### Attacker's View

A perspective included in each TLCTC threat cluster definition that describes how the attacker perceives or approaches the exploitation of the specific generic vulnerability. It helps distinguish between clusters by focusing on the attacker's methodology rather than technical implementation details.

## Attack Path

The sequence of applied Attack Vectors in a cyber incident, representing an ordered sequence of Attack Steps describing a complete attack scenario. Basic notation uses  $\#X \rightarrow \#Y \rightarrow \#Z$  (e.g.,  $\#9 \rightarrow \#3 \rightarrow \#7$ ). Attack paths may include velocity annotations showing the time between steps (e.g.,  $\#9 \rightarrow [24h] \#4 \rightarrow [12m] \#1$ ), domain boundary markers using the  $|||$  operator, parallel steps, and Data Risk Event tags.

**Reference:** §2.2.2 (Global Definitions), §3.0 (Path Semantics)

## Attack Path Notation

The standardized format for describing cyber attack sequences using TLCTC clusters. Format uses:  $\rightarrow$  for sequential steps,  $+$  for parallel execution,  $[time]$  for temporal intervals, and  $||| [context] [ @Source \rightarrow @Target ] |||$  for domain boundaries. Example:  $\#9 \rightarrow [24h] \#4 \rightarrow [12m] \#1 ||| [dev] [ @Vendor \rightarrow @Org ] ||| \rightarrow [weeks] \#10.2 \rightarrow [0s] \#7$ .

## Attack Sequence Schema (V2.0)

The JSON schema that defines the required structure for documenting attack path instances. The schema ensures all documented attacks follow a consistent format including metadata (sequence\_id, attack\_title, framework\_version), temporal transitions (delta\_t\_value, delta\_t\_unit, velocity\_class), responsibility spheres, and cluster mappings. File format: `tlctc-attack-sequence-schema.json`.

## Attack Step (V2.0)

A single attacker action or event that exploits exactly **one generic vulnerability** in a specific context. Each Attack Step MUST map to exactly one TLCTC cluster (per Axiom VI).

**Reference:** §2.2.2 (Global Definitions), §2.2.7 (Minimal Classification Procedure)

## Attack Vector

The specific path or method used by an attacker to gain unauthorized access to a target system. In the TLCTC framework, each distinct attack vector is a distinct initiating method defined by the **initial generic vulnerability targeted** (per Axiom VII). The vector label MUST be based on cause, not outcome.

**Reference:** §2.2.2 (Global Definitions), Axiom VII (§1.2)

## Attack Velocity ( $\Delta t$ ) (V2.0)

The temporal dimension of cyber risk representing the **time interval** between two adjacent Attack Steps in an attack path. For an edge  $\#X \rightarrow \#Y$ , the value  $\Delta t(X \rightarrow Y)$  represents the elapsed time between step  $\#X$  and step  $\#Y$  in the described scenario.  $\Delta t$  is an edge property attached to the sequence operator, not to steps. Attack velocity is the single most accurate

predictor of attacker sophistication and the only metric that truthfully measures control effectiveness. Categorized into four velocity classes: Latent/Slow (days to months), Medium (hours), Fast (minutes), and Realtime (seconds/milliseconds).

**Reference:** §4.0 (Definitions), §4.1 (Measurement Model), §4.2 (Notation)

## Availability (Data Risk Event)

The technical state in which data or resources exist and can be reached by the infrastructure. Availability answers the question: "Does the data exist and can the system access it?" A resource is available if it is present, can be enumerated, and is technically accessible to the system—regardless of whether it can be meaningfully used. For example, encrypted files remain available (they exist on disk) even when rendered unusable by ransomware. Loss of Availability occurs when data is deleted, storage fails, or systems go offline. DRE notation:  $\boxed{Av}$  when distinguished from Accessibility, or  $\boxed{A}$  as general shorthand covering both.

## Axiom

A foundational premise that defines what terms mean and what kinds of statements are allowed in TLCTC. Axioms are non-negotiable constraints on interpretation that force methodological consequence and prevent logical shortcuts and category errors. These foundational principles must be accepted to validate and effectively use the TLCTC framework. TLCTC defines ten axioms organized into four groups: Scope (I–II), Separation (III–V), Classification (VI–VIII), and Sequence (IX–X).

**Reference:** §1.2 (Axioms and Assumptions)

## B

### Bounded $\Delta t$

A minimum or maximum bound for  $\Delta t$  derived from known constraints when precise timestamps are unavailable. Notation:  $\boxed{\Delta t < 15m}$  (upper bound),  $\boxed{\Delta t > 15m}$  (lower bound),  $\boxed{\Delta t = 10m \dots 20m}$  (range).

**Reference:** §4.0.3, §4.2.3

## Bow-Tie Model

A risk model that represents risk as a structure with five elements: Threats (left side), Preventive Controls (left side), Central Event (knot), Mitigating Controls (right side), and Consequences (right side). TLCTC is anchored in the Bow-Tie model to enforce strict separation between cause and effect in cyber risk analysis. The model enforces temporal causality, prevents confusion between threats and outcomes, enables precise control placement, and reveals attack sequences as causal chains. The central event "Loss of Control" serves as the pivot point between threat realization and potential consequences.

**Reference:** §1.4 (The Bow-Tie Anchor), §1.4.1 (Structure and Vocabulary)

## Bridge Cluster

A TLCTC cluster whose generic vulnerability **inherently** enables crossing into (or leveraging over) a different domain's control regime. Bridge clusters are: #8 Physical Attack, #9 Social Engineering, and #10 Supply Chain Attack.

**Reference:** §2.2.2 (Global Definitions), §5.1.4, §5.2 (Topology Classification)

## Bridge Step

A step-level instance of a bridge cluster that crosses a specific domain boundary. When a bridge step crosses responsibility spheres, the boundary **SHOULD** be recorded in path notation via the domain boundary operator `|| [context] [ @Source→@Target ] ||`.

**Reference:** §2.2.2 (Global Definitions), §5.1.6

## BxIs (Base Level Indicators)

The lowest level of indicators that still make operational sense, representing metrics at the operational level directly translated into measurable values. Part of the hierarchical KxI framework (KRIs, KCIs, KPIs).

# C

## Capacity Exhaustion

Degradation or denial of service caused **primarily** by volume or intensity exceeding finite resources. Resources include: bandwidth, CPU cycles, memory, storage, database connections, API quotas, thread/process pools, file handles. Maps to #6 Flooding Attack.

**Reference:** §2.2.2 (Global Definitions), R-FLOOD (§2.2.4)

## Central Event

In the TLCTC Bow-Tie model: **Loss of Control / System Compromise** — the point at which the attacker achieves unauthorized control over the system's behavior, privileges, data, or trust relationships—sufficient to pursue attack objectives. This central event is positioned before outcomes.

**Reference:** §1.4.3 (Central Event)

## Client-Role Component

A component that **consumes external responses, content, or state** relative to the attacker. The component is in "client role" for the specific interaction being classified.

**Reference:** §2.2.2 (Global Definitions), R-ROLE (§2.2.4)

## **Client-Server Relationship**

A fundamental principle (Axiom II) stating that every networked software system is based on client-server or caller-called function interaction at various levels. The relationship is contextual: the entity requesting a service is the "client," and the entity providing that service is the "server". Roles can be dynamic and change depending on interaction context, particularly across protection ring boundaries.

## **Coder**

A development role focused on implementation and craftsmanship, responsible for writing functional, efficient code according to established patterns, implementing specific security controls at the code level, and following secure coding practices. Primary responsibility for addressing threat clusters #2, #3, and implementation details of #4, #5, and #7. Contrasts with the Programmer role which focuses on architecture and strategy.

## **Consequences**

In the Bow-Tie model: what results after the central event, including technical and business impact (event chains). Consequences are on the right (effect) side of the Bow-Tie and are recorded as Data Risk Events. Consequences are NOT threat categories.

**Reference:** §1.4.1 (Bow-Tie Structure), §1.4.4 (What TLCTC Does NOT Classify)

## **Control**

A security measure implemented to mitigate threats, reduce vulnerabilities, or minimize the impact of security incidents. In the TLCTC framework, controls are organized using NIST CSF functions (Identify, Protect, Detect, Respond, Recover) and mapped to specific threat clusters. Controls are categorized as Local Controls (protecting specific systems) or Umbrella Controls (protecting groups of systems).

## **Control Design Effectiveness**

An evaluation of whether a control, as conceived and structured, is theoretically capable of achieving its objective if it operates as intended. Assesses the control's capability to address the identified risk within its specific threat cluster.

## **Control Failure**

A deviation from a control objective or lack of effectiveness. Control failure is control-risk and MUST NOT be treated as a threat category (Axiom V). Risk structure remains: Threat → Event/Incident → Consequences; controls influence likelihood and impact but do not define the threat cluster. Distinguished from the actual risk event itself (Axiom IV).

**Reference:** Axiom V (§1.2), §1.4.2 (Rule 3)

## Control Objective

The specific aim or purpose that a control is intended to achieve, defining what the control should accomplish in terms of risk mitigation for a particular threat cluster. Each control aligns with a single, clear objective.

## Control Operational Effectiveness

An evaluation of whether a control is actually working as designed in practice, examining if the control is being executed correctly and consistently over time to meet its objective. May vary depending on the nature of the threat cluster (e.g., controls for Malware #7 may never achieve 100% due to detection latencies).

## Credential / Identity Artifact

Any secret, token, key, or session artifact that enables authentication or authorization decisions. Examples include: passwords, PINs, passphrases, API keys, bearer tokens, OAuth/OIDC tokens, SAML assertions, session cookies, session identifiers, private keys, client certificate keys, Kerberos tickets, SSH keys, hardware token seeds/OTPs, biometric templates (when used as authenticators).

**Reference:** §2.2.2 (Global Definitions), Axiom X (§1.2)

## Credential Acquisition

The act of obtaining, capturing, exposing, deriving, or forging a credential/identity artifact. Credential acquisition maps to the **enabling cluster**—the generic vulnerability that made the acquisition possible.

**Reference:** §2.2.2 (Global Definitions), R-CRED (§2.2.4), Axiom X (§1.2)

## Credential Application

The act of presenting, using, replaying, or leveraging a credential to authenticate and operate as an identity. Credential application MUST always map to #4 Identity Theft.

**Reference:** §2.2.2 (Global Definitions), R-CRED (§2.2.4), Axiom X (§1.2)

## Credential Forgery

The act of creating a credential without possessing the legitimate secret. If forgery succeeds due to an implementation flaw (e.g., weak signing algorithm, missing validation, predictable tokens), the forgery step maps to #2 or #3 per R-ROLE. The subsequent use of the forged credential maps to #4.

**Reference:** §2.2.2 (Global Definitions), R-CRED (§2.2.4)

## **CVE (Common Vulnerabilities and Exposures)**

A standardized identifier for publicly known cybersecurity vulnerabilities. In the TLCTC framework, CVEs are mapped to generic vulnerabilities and their corresponding threat clusters to enable consistent threat classification and control implementation.

## **Cyber Bow-Tie**

The specific application of the Bow-Tie Model to cyber risk management, with the 10 Top Level Cyber Threat Clusters on the cause side, "Loss of Control" or "System Compromise" as the central event, and Data Risk Events and Business Risk Events on the consequence side. Enables structured cyber risk register development and event chain analysis.

## **Cyber Incident**

An actual security breach or system compromise that has occurred, representing the materialization of a cyber risk event where control over IT systems or persons has been lost due to one or more of the 10 Top Level Cyber Threat Clusters.

## **Cyber Risk**

The probability of occurrence of a cyber event in which control over IT systems or persons is lost due to one or more of the 10 Top Level Cyber Threat Clusters, leading (via event chains) to consequential damage (impact). Cyber risks are a subset of operational risks (OpRisk).

## **Cyber Risk Event**

A potential occurrence that could lead to a system breach or compromise. Distinguished from Cyber Incidents (which have already occurred) and Data Risk Events (which are consequences). The central event in the Cyber Bow-Tie model is "Loss of Control" or "System Compromise".

## **Cyber Threat Radar**

A visualization tool based on the TLCTC framework that displays threat distribution across different domains (organizational, state, or sector levels). Uses radar chart format to show impact levels (High/Red, Medium/Orange, Low/Gray, Latent) and movement indicators (▲ increasing, ▼ decreasing) for each of the 10 threat clusters. Enables strategic overview, comparative analysis, and standardized threat communication across organizations and borders.

## D

### Data Processing Pathways

The four distinct paths that data can follow during an attack, each mapping to specific TLCTC clusters:

1. Data → Data (#1 only) — Pure data manipulation without code execution
2. Data → Function Invocation → Foreign Code Execution (#1 → #7) — Function abuse enabling code execution
3. Data → Exploit Code via Implementation Flaw (#2 or #3) — Unintended data→code transition
4. Data → Foreign Code via Designed Execution Capability (#7 only) — Intended execution capability

### Data Risk Event (DRE)

An outcome event describing **Loss of Confidentiality (C)** (data stolen / unauthorized access), **Loss of Integrity (I)** (data modified / unauthorized changes), or **Loss of Availability/Accessibility (A)** (data gone or unreachable, or data present but unusable). Data Risk Events MUST be recorded separately from cluster steps, MUST NOT be used as threat categories, and MUST NOT change the cluster classification of the step that preceded them. Notation: `[DRE: C]`, `[DRE: I]`, `[DRE: A]`, or combinations. When the distinction between Availability and Accessibility is operationally relevant, the general code `A` MAY be refined into `Av` (Availability — data gone or unreachable) or `Ac` (Accessibility — data present but unusable). Example: ransomware encryption = `[DRE: Ac]`; data deletion = `[DRE: Av]`; distinction unknown = `[DRE: A]`.

**Reference:** §2.2.2 (Global Definitions), §1.4.2 (Rule 2), §3.5.3

### Data vs Code Boundary

A normative classification principle: Domain-specific expressions (e.g., SQL, LDAP, XPath, GraphQL, template syntax, configuration languages) are treated as **data** unless they directly cause **FEC execution** via a general-purpose execution engine.

**Reference:** §2.2.2 (Global Definitions)

### Delta t ( $\Delta t$ ) (V2.0)

Symbol representing the time interval between threat cluster transitions in an attack sequence. See Attack Velocity.



## Designed Execution Capability

The environment's **intended** capability to load, interpret, or execute program content. This is the generic vulnerability exploited by **#7 Malware**. Examples: OS loaders, script interpreters, macro engines, browser JS engines, module loaders, container/virtualization runtimes.

**Reference:** §2.2.2 (Global Definitions), §2.1 (#7 Definition)

## Detection Coverage Score (DCS) (V2.0)

A strategic Key Performance Indicator (KPI) for measuring security effectiveness derived from Attack Velocity. Formula:  $DCS = \frac{\text{Mean Time to Detect}}{\text{Attack Velocity } \Delta t}$ .

- **Score < 1.0:** Organization is faster than the adversary (Winning)
- **Score > 1.0:** Adversary completes the step before detection (Losing)

Example: If a ransomware group moves from #4 to #1 in 10 minutes and your SIEM alerts in 15 minutes,  $DCS = 15/10 = 1.5$ , indicating systematic blindness requiring automation rather than analyst intervention.

## Developer's View

A perspective included in each TLCTC threat cluster definition that provides guidance on secure development practices specific to preventing that cluster. Encompasses both Programmer (architectural) and Coder (implementation) responsibilities.

## Domain

A set of assets governed by a coherent control regime (policies, monitoring, enforcement, and accountability). Domains may be technical, organizational, or socio-technical. Examples: cyber/IT domain, physical security domain, human decision domain, vendor development domain, cloud provider control-plane domain.

**Reference:** §2.2.2 (Global Definitions), §5.1.1

## Domain Boundary

A point where responsibility spheres or control regimes change. Crossing a domain boundary means the attack moves from one set of applicable controls to a different set.

**Reference:** §2.2.2 (Global Definitions), §5.1.3

## Domain Boundary Operator (||) (V2.0)

Notation: `||[context][@Source→@Target]||`. Used to explicitly mark where an attack path crosses responsibility spheres. The operator SHOULD accompany bridge cluster steps (`#8`, `#9`, `#10`) and MAY be used with any step that crosses a domain boundary. The context describes the transition type (e.g., `[dev]`, `[idp]`, `[update]`) and the arrow shows the direction of trust crossing. The boundary test: "If removing the third-party trust link would stop the step from succeeding, #10 belongs there". Enables precise mapping of responsibility shifts and supply chain attack analysis.

**Reference:** §2.2.2 (Global Definitions), §3.3 (Domain Boundary Operator), §5.3

## Dual-Use Tool

A legitimate administrative utility that can be used for both legitimate administrative purposes and malicious activities when invoked by an attacker. Examples include PowerShell, PsExec, WMI, and remote administration tools. In TLCTC: invocation/abuse of the tool may be `#1` (if no implementation flaw is exploited), while the actual execution of attacker-controlled FEC through that tool is `#7`, resulting in a `#1 → #7` sequence.

**Reference:** §2.2.4 (R-EXEC, LOLBAS Clarification)

## E

### Edge (in attack path)

A transition between two adjacent Attack Steps, represented by the sequence operator `→`.  $\Delta t$  (Attack Velocity) is an edge property.

**Reference:** §3.1 (Sequence Operator), §4.0.2

### Estimated $\Delta t$

An approximate  $\Delta t$  value derived from partial evidence when precise timestamps are unavailable. Notation:  `$\Delta t \sim 15m$` .

**Reference:** §4.0.3, §4.2.3

## E<sub>n</sub> Event Notation (Regulatory) (V2.0)

A numbered event sequence notation used to map attack chains to regulatory compliance triggers:

- **E1:** System Compromise / Loss of Control (the central Bow-Tie event)
- **E2:** Data Risk Event (e.g., PII exposure — GDPR trigger)
- **E3a, E3b, ...:** Compliance violation events (e.g., GDPR breach notification, NIS2 incident report)

The subscript (a, b, etc.) distinguishes parallel regulatory branches triggered by the same upstream event. Different regulations trigger at different points: GDPR Art. 33 triggers at E2 (Data Risk Event involving PII), while NIS2 Art. 23 triggers at E1 (Significant Incident). See also: Event Chain Length, RS Container.

### **Event Chain Length (V2.0)**

The number of causal events between the initial incident (E1) and a regulatory trigger point (E3x). Shorter chains mean compliance clocks start sooner. Example: NIS2 path (E1→E3b) = 2 events with 24h early warning requirement; GDPR path (E1→E2→E3a) = 3 events with 72h notification timeline. Understanding chain length helps CISOs structure incident response playbooks to meet multiple regulatory timelines from a single incident.

### **Exploit Code**

Foreign code that targets specific vulnerabilities to modify software behavior, creating an UNINTENDED data→code transition. Used in #2 Exploiting Server and #3 Exploiting Client. Distinguished from Malware Code which operates within expected execution paths.

### **Exploiting Client (#3)**

A threat cluster where an attacker targets and leverages flaws originating directly within the source code implementation of any software acting in a client role (requesting/processing data from a server or resource). These vulnerabilities allow manipulation of client behavior or unauthorized access using Exploit Code, often when the client interacts with malicious content. The generic vulnerability is the presence of exploitable flaws within client-side source code stemming from insecure coding practices.

### **Exploiting Server (#2)**

A threat cluster where an attacker targets and leverages flaws originating directly within the server-side application's source code implementation. These vulnerabilities allow manipulation of server behavior or unauthorized access using Exploit Code, forcing a data→code transition where exploit code executes as new, foreign code in the server context. The generic vulnerability is the presence of exploitable flaws within server-side source code implementation stemming from insecure coding practices.

## **F**

### **Fast Velocity Class (V2.0)**

A velocity classification where attack progression occurs within minutes. Typical threat clusters: #3 (Exploiting Client), #2 (Exploiting Server). Control strategy requires automated containment and EDR blocking, as human analyst response times are insufficient. Example: Drive-by downloads, browser exploits, RCE exploitation.

## Flooding Attack (#6)

A threat cluster where an attacker intentionally overwhelms system resources or exceeds capacity limits through a high volume of requests, data, or operations, leading to disruption, degradation, or denial of service for legitimate users. The generic vulnerability is the finite capacity limitations inherent in any system component (network bandwidth, CPU, memory, storage, database limits, application quotas, API rate limits, process/thread pools). Outcome is typically Loss of Availability.

## Foreign Executable Content (FEC)

Attacker-controlled (or otherwise untrusted) program text or bytes that are **interpreted, loaded, or executed** by a **general-purpose execution engine** in the target environment. Includes attacker-controlled commands fed into interpreters. FEC execution includes in-memory (fileless) execution, interpreted code, macro execution, and reflective loading—no "on-disk" requirement exists.

**Reference:** §2.2.2 (Global Definitions)

## Framework Layer (V2.0)

The static, universal component of the TLCTC JSON architecture containing threat cluster definitions, generic vulnerabilities, data risk events, bow-tie model principles, attack path notation rules, and framework axioms. Defined in `tlctc-framework.json`. Changes rarely (only during framework evolution) and serves as the common language that all organizations reference. Contrasts with the Intelligence Layer which contains dynamic, incident-specific data.

# G

## Generic Vulnerability

The single root-level vulnerability category defining a cluster. For every generic vulnerability, there is exactly one TLCTC cluster (per Axiom VI). Generic vulnerabilities are stable across technologies and implementations, persisting regardless of specific IT system types, software implementations, or evolving attack techniques. The 10 generic vulnerabilities are: functional scope/trust (#1), server-side implementation flaws (#2), client-side implementation flaws (#3), identity-artifact binding (#4), lack of end-to-end communication protection (#5), finite capacity limitations (#6), designed execution capability (#7), physical accessibility (#8), human psychological factors (#9), and third-party trust dependencies (#10).

**Reference:** §2.2.2 (Global Definitions), §2.2.7 (Step 2), Axiom VI (§1.2)

## GOVERN (GV)

The governance function in NIST CSF 2.0, operating at a strategic level to establish the overall cybersecurity risk management framework. GOVERN controls are "assurance controls" that create structure and context for other functions, including setting risk appetite, defining roles and responsibilities, and establishing policies. While GOVERN oversees threat categorization in the risk register, GV controls themselves don't directly counter specific threats but provide the strategic foundation for comprehensive risk management.

## I

### Identity Theft (#4)

A threat cluster where an attacker targets weaknesses in identity and access management processes or credential protection mechanisms to illegitimately misuse authentication credentials (passwords, tokens, keys, session identifiers, biometrics) to impersonate a legitimate identity (human or technical). The generic vulnerability is weak Identity Management Processes and/or inadequate credential protection mechanisms throughout the identity lifecycle.

**Critical distinction:** Credentials have dual operational nature:

- **Acquisition/Exposure:** When credentials are obtained through another cluster (e.g., #2 SQL injection, #5 MitM, #7 keylogger, #9 Phishing), map to the enabling cluster (Loss of Confidentiality consequence)
- **Use/Application:** The subsequent *use* of acquired credentials—regardless of acquisition method—always maps to #4 Identity Theft (Loss of Control / system compromise event)

Non-Overlap Rule: Credential acquisition maps to the enabling threat cluster; credential use always maps to #4.

### Implementation Defect (Availability Context)

A flaw in code logic, parsing, memory handling, or resource handling that causes crash, hang, or degradation when triggered—**without** requiring volume/intensity to exceed normal capacity. Includes algorithmic complexity weaknesses (e.g., ReDoS). Maps to #2 or #3 per R-ROLE, not #6.

**Reference:** §2.2.2 (Global Definitions), R-FLOOD (§2.2.4)

### Implementation Flaw

A defect in source code implementation (logic, parsing, memory handling, resource handling) enabling unintended behavior when triggered. Implementation flaws are exploited by #2 Exploiting Server (server-role) or #3 Exploiting Client

(client-role).

**Reference:** §2.2.2 (Global Definitions), §2.1 (#2 and #3 Definitions)

## Intelligence Layer (V2.0)

The dynamic component of the TLCTC JSON architecture containing specific attack instances, software versions & CVEs, timeline & actor TTPs, domain boundaries, and impact assessments. Changes constantly as new incidents occur. Each incident is documented in its own JSON file following the attack sequence schema format: `[incident-id]-attack-path.json`. Enables worldwide threat intelligence sharing while maintaining consistency through reference to the static Framework Layer.

## Internal Cluster

A TLCTC cluster that operates primarily **within the software domain's** attack surfaces, without inherently crossing to a different responsibility sphere. Internal clusters are: #1 through #7.

**Reference:** §2.2.2 (Global Definitions), §5.1.5, §5.2 (Topology Classification)

## Intra-System Boundary Operator (|...|) (V2.1)

Notation: `|[type][@from→@to]|`. Used to annotate boundary crossings **within a single host or system**, such as sandbox escapes, privilege escalations, process boundary violations, and VM escapes. Uses single pipe delimiters to distinguish from the inter-sphere Domain Boundary Operator (`||...||`). Defined boundary types: `sandbox`, `privilege`, `process`, `hypervisor`. The `memory` type is reserved and MUST NOT be used (R-INTRA-9). Intra-system boundaries are observability annotations and never change cluster classification (R-INTRA-7). Example: `#3 |[sandbox][@renderer→@os]|` — browser exploit escaping renderer sandbox.

**Reference:** §3.3.6 (Intra-System Boundary Operator)

# J

## JSON Architecture (V2.0)

The standardized data structure for threat intelligence sharing in TLCTC V2.0, consisting of four complementary JSON files:

1. **tlctc-framework.json:** Core framework definitions (universal, rarely updated)
2. **tlctc-responsibility-spheres.json:** Domain boundary definitions (customizable, occasionally updated)
3. **tlctc-attack-sequence-schema.json:** Validation schema for attack instances (universal, rarely updated)

4. **[incident]-attack-path.json**: Specific attack instances (per-incident, constantly updated)

This architecture separates universal framework definitions from specific attack instances, enabling machine-readable, validated, consistent threat intelligence exchange worldwide.

## K

### **KCI (Key Control Indicator)**

A metric that measures the operational performance of security controls, verifying that intended actions are taken at the appropriate frequency. KCIs provide insights on the ability to apply correct controls correctly, highlighting process weaknesses and tool effectiveness. Example: "Frequency of patch deployments per day" or "Scan verification of implemented patches" for a control requiring critical systems to be patched within 24 hours.

### **KPI (Key Performance Indicator)**

A measurable value demonstrating the outcome and performance of security processes in reaching security objectives. KPIs must be time-based and reflect effectiveness over time. Example: "Average time to restore critical services to full operation within a 4-hour window." In TLCTC V2.0, the Detection Coverage Score (DCS) is introduced as a strategic KPI.

### **KRI (Key Risk Indicator)**

A leading indicator demonstrating the potential for a future cyber threat. KRIs show possible risks before a threat occurs and must be observed in a meaningful timeframe. Example: "Number of unpatched critical vulnerabilities older than 7 days" indicates how processes handle critical vulnerabilities, helping identify, understand, and prioritize security efforts to prevent incidents.

### **KxI Framework**

The integrated hierarchical framework of Key Risk Indicators (KRIs), Key Control Indicators (KCIs), and Key Performance Indicators (KPIs), providing a practical mechanism to operationalize the 10 Top Level Cyber Threat Clusters. Each threat cluster has associated KRI, KCI, and KPI values for managing cyber risk and measuring overall cybersecurity program performance. Base Level Indicators (BxIs) represent the lowest operational level that still makes sense.

# L

## Latent/Slow Velocity Class (V2.0)

A velocity classification where attack progression occurs over days to months. Typical threat clusters: #10 (Supply Chain), #7 (APT Implants). Control strategy focuses on log retention and threat hunting, as detection windows are extended. Example: Supply chain compromises with long dwell times, persistent APT campaigns prioritizing stealth over speed.

## Living Off the Land / LOLBAS (Living Off the Land Binaries and Scripts)

An attack technique using only software functions and binaries already present on a (potentially compromised) system, invoked with legitimate inputs/parameters, without introducing foreign code initially. Legitimate system binaries are used to execute attacker-controlled content. In TLCTC: the invocation of the legitimate binary may be #1 (if no implementation flaw is exploited), while the execution of attacker-controlled content through it is #7. The sequence #1 → #7 applies. Examples: Using cmd.exe, PowerShell, WMI, or Task Scheduler to execute attacker-controlled scripts.

**Reference:** §2.2.4 (R-EXEC, LOLBAS Clarification)

## Local Controls

Security measures implemented directly on or for specific IT systems. Distinguished from Umbrella Controls which protect groups of systems. Local controls are essential for systems that cannot be fully protected by umbrella controls (e.g., exposed systems, "Patient Zero" entry points).

## Loss of Availability (LoA)

A Data Risk Event outcome where data or resources are gone or unreachable — the resource no longer exists or cannot be technically accessed by the infrastructure. From the attacker's perspective: "Data gone / system down". Examples: deletion, storage failure, system offline, network unreachable. DRE notation: **Av** (refined) or **A** (general). Not to be confused with Loss of Accessibility (data present but unusable, e.g., ransomware encryption → **Ac**).

## Loss of Accessibility (LoAc)

A Data Risk Event outcome where data or resources exist and can be reached by the infrastructure, but cannot be used for their intended purpose by authorized processes. From the attacker's perspective: "Data locked / unusable". Examples: ransomware encryption, data corruption, permission lockout. DRE notation: **Ac** (refined) or **A** (general). Not to be confused with Loss of Availability (data gone or unreachable → **Av**).



## Loss of Confidentiality (LoC)

A Data Risk Event outcome where an attacker gains unauthorized access to data. From the attacker's perspective: "Data stolen". This describes what bad thing happens, not how it happens. Various threat clusters can lead to this outcome depending on the mechanism used (e.g., #2 via SQL injection, #5 via MitM eavesdropping).

## Loss of Control / System Compromise

The central event in the Cyber Bow-Tie model, representing the point at which the attacker achieves unauthorized control over a system's behavior, privileges, data, or trust relationships. This serves as the pivot point between threat realization (cause) and potential consequences (effect). Some attacks may have delayed data risk events (creating a detection window), while others lead to immediate data risk events. Examples: A server exploit (#2) enabling remote code execution leading to malware (#7) represents loss of control before any data breach occurs. In contrast, successful SQL injection (#2) can immediately result in Loss of Confidentiality.

**Reference:** §1.4.3 (Central Event)

## Loss of Integrity (LoI)

A Data Risk Event outcome where an attacker successfully makes unauthorized changes to data. From the attacker's perspective: "Data modified". This refined definition focuses on the outcome rather than the mechanism (e.g., "tampering" is a mechanism that produces the LoI outcome).

# M

## Malicious Code

Code written with harmful intent, distinguished in TLCTC between:

- **Exploit Code:** Targets specific vulnerabilities to modify software behavior (#2/#3)
- **Malware Code:** Operates within expected execution paths for harmful purposes (#7)
- **Malware Software:** Comprehensive suite of tools (foreign code) that may incorporate multiple techniques, including exploit capabilities

## Malware (#7)

A threat cluster where an attacker abuses the inherent ability of a software environment to execute foreign executable content, including inherently malicious Malware Code or legitimate tools/scripts when they execute attacker-controlled or otherwise foreign code ("dual-use"). The generic vulnerability is the software environment's designed capability to

execute potentially untrusted 'foreign' code, scripts, or binaries. Distinguished from #2/#3 which use Exploit Code targeting implementation flaws, and from #1 which manipulates existing functions without executing foreign code/scripts/binaries.

### **Man in the Middle (#5)**

A threat cluster where an attacker intercepts, eavesdrops on, modifies, or relays communication between two parties without their knowledge or consent, by exploiting a privileged position on the communication path. The generic vulnerability is the lack of sufficient control, integrity protection, or confidentiality over the communication channel/path, including the implicit trust placed in local networks and intermediate network infrastructure in standard IP networking. Position might be gained locally (shared Wi-Fi) or by leveraging control over existing network intermediaries.

### **Medium Velocity Class (V2.0)**

A velocity classification where attack progression occurs within hours. Typical threat clusters: #9 (Phishing), #4 (Manual Credential Abuse). Control strategy utilizes SIEM alerting and analyst triage, as human response times are sufficient for detection and initial response. Example: Phishing campaigns followed by manual reconnaissance and lateral movement.

### **Mitigating Controls**

In the Bow-Tie model: barriers on the right (effect) side that detect, contain, reduce impact, or enable recovery after the central event occurs. Corresponds to NIST CSF functions: RESPOND, RECOVER.

**Reference:** §1.4.1 (Bow-Tie Structure)

### **MITRE ATT&CK**

A globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. In the TLCTC framework, MITRE techniques are considered operational-level detail that map to the strategic-level threat clusters. TLCTC V2.0 proposes enhancement through adding cluster mappings and typical velocity attributes to techniques.

### **MitM Position**

A controlled point on a communication path that enables interception, observation, modification, injection, replay, or protocol downgrade/stripping. The attacker has achieved the ability to influence communication between two endpoints.

**Reference:** §2.2.2 (Global Definitions), R-MITM (§2.2.4)

## N

### NIST CSF (Cybersecurity Framework)

The National Institute of Standards and Technology Cybersecurity Framework providing guidelines for managing cybersecurity risk. The TLCTC framework integrates with NIST CSF by mapping the 10 threat clusters to the five core functions (Identify, Protect, Detect, Respond, Recover) and the GOVERN function in CSF 2.0. TLCTC proposes formal adoption of the 10 clusters as the standard taxonomy for Threat Identification in the ID.RA (Risk Assessment) category.

### Normative Keywords

The keywords MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL in the TLCTC specification are interpreted as described in RFC 2119 / RFC 8174. When these keywords appear in lowercase, they carry their ordinary English meaning.

**Reference:** §2.2

### Notation Systems

The TLCTC framework employs two complementary notation systems:

- **Strategic Notation:** Human-readable format using `#X` (e.g., #1, #10) for executive communication and risk assessment
- **Operational Notation:** Machine-readable format using `TLCTC-XX.YY` (e.g., TLCTC-01.00) for tool integration, automation, and SIEM

Both notations remain fully compatible and can be used interchangeably based on context.

## O

### Observed $\Delta t$

A  $\Delta t$  value computed from two concrete time observations.

**Reference:** §4.0.3

### Operational Layer

The detailed implementation level where security controls are implemented, monitored, and adjusted. Includes specific vulnerability management, threat intelligence (using frameworks like MITRE ATT&CK), TTP mapping, attack path analysis, vulnerability management (CVE reports), incident response, security testing, and monitoring. Uses the

machine-first naming convention `TLCTC-XX.YY`, where XX is the two-digit cluster number (01–10) and YY is the two-digit sub-cluster number (00–99), for tool integration, SIEM rules, automation, threat intelligence exchange, and detailed documentation.

**Reference:** §2.2.1 (Two-Layer Naming Convention)

## Operational Security Layer

The layer of TLCTC that addresses specific vulnerabilities, techniques, and procedures. Contains concrete vulnerabilities (CVEs), operational techniques (TTPs), and indicators used in detection, response, and engineering. Corresponds to `TLCTC-XX.YY` where YY ≠ 00.

**Reference:** Axiom VIII (§1.2), §2.2.1

## P

### Parallel Operator (+)

Notation that denotes **concurrent** (or effectively concurrent) steps—actions that occur in the same phase where their ordering is not meaningful. Parallel steps **MUST** be grouped using parentheses: `(#X + #Y)`. Each element inside a parallel group is a separate Attack Step mapped to exactly one cluster.

**Reference:** §2.2.2 (Global Definitions), §3.2 (Parallel Operator)

### Parallel Steps

Two or more clusters occurring simultaneously or in tight coordination within the same attack phase. Use when distinct generic vulnerabilities are exploited concurrently rather than sequentially.

**Reference:** §2.2.2 (Global Definitions), §3.2 (Parallel Operator)

### Physical Attack (#8)

A threat cluster where an attacker gains unauthorized physical interaction with or causes physical interference to hardware, devices, facilities, or data transmission media (including wireless signals). The generic vulnerability is the physical accessibility of hardware, facilities, and communication media, and the exploitability of Layer 1 (Physical Layer) communications and hardware interfaces. Encompasses two main types:

- **Direct Physical Access Attacks (#8.1):** Require physical touch or direct interaction (tampering, theft, physical intrusion, unauthorized device connection)
- **Indirect Physical Access Attacks (#8.2):** Exploit physical properties without direct contact (TEMPEST, signal jamming, acoustic attacks, environmental disruption)

## Position Acquisition vs Position Exploitation

For #5 Man in the Middle: **Gaining** a MitM position maps to another cluster (#1, #8, #9, #10, or #2/#3 depending on initial generic vulnerability). **Exploiting** a MitM position (intercept, modify, relay, inject, replay, downgrade actions) maps to #5.

**Reference:** §2.2.2 (Global Definitions), R-MITM (§2.2.4)

## Preventive Controls

In the Bow-Tie model: barriers on the left (cause) side that reduce likelihood of threats reaching the central event. Corresponds to NIST CSF functions: IDENTIFY, PROTECT.

**Reference:** §1.4.1 (Bow-Tie Structure)

## Programmer

A development role focused on architecture and strategy, responsible for designing overall software architecture and component interactions, making strategic decisions about frameworks and protocols, establishing secure coding standards and security requirements, and considering system-wide security implications. Primary responsibility for addressing threat clusters #1, #4, #5, #10 at an architectural level. Contrasts with the Coder role which focuses on implementation and craftsmanship.

## Propagated PR (V2.0)

A Protection Requirement that "propagates backward" from a downstream event into the RS (Respond) container of an earlier event due to regulatory or policy requirements.

Notation:  $RS(E_n) = \{ \text{Response} \} \cup \{ \text{Propagated PR}(E_{n+1}) \} \cup \{ \text{Propagated PR}(E_{n+x}) \}.$

This mechanism explains how multiple regulatory obligations stack into a single incident response workflow. Example: A ransomware attack triggers E1 (System Compromise). If PII is affected, E2 (Data Risk Event) occurs, propagating GDPR notification requirements back into RS(E1). Simultaneously, if the organization is NIS2-scoped, the incident itself propagates NIS2 reporting into RS(E1). The result: two separate Propagated PR controls in the same RS container, with different timelines (72h vs 24h+72h) and different authorities. See also: RS Container, E<sub>n</sub> Event Notation.

## Protection Ring Architecture

The layered privilege model in computing systems (Ring 0 through Ring 3) where each ring represents a different privilege level. TLCTC framework analyzes threats at ring boundary interactions: Ring 0 (Kernel Mode), Ring 1 (HAL/Driver Level), Ring 2 (OS Services), Ring 3 (User Mode). Nine threat clusters (excluding #9 Social Engineering) apply at each boundary, with roles (client/server) determined by the direction of interaction across rings.

# R

## R-ABUSE (Function Misuse Determination)

Global mapping rule: If the attacker's success does not require any implementation flaw and instead abuses intended functionality, scope, or configuration via standard interfaces using expected input types, the step MUST be classified as **#1 Abuse of Functions**.

**Reference:** §2.2.4 (R-ABUSE)

## R-CRED (Credential Lifecycle Non-Overlap)

Global mapping rule: Credential acquisition maps to the enabling cluster; credential application MUST always map to **#4 Identity Theft**. If both occur, they MUST be represented as at least two steps: **(enabling cluster) → #4**.

**Reference:** §2.2.4 (R-CRED)

## R-EXEC (Foreign Execution Recording Rule)

Global mapping rule: Whenever Foreign Executable Content (FEC) is interpreted, loaded, or executed, a **#7 Malware** step MUST be recorded at the moment of execution, independent of how execution was enabled. **#7** is additive (does not replace the enabling cluster).

**Reference:** §2.2.4 (R-EXEC)

## R-FLOOD (Capacity Exhaustion vs Implementation Defect)

Global mapping rule: If the primary mechanism is volume or intensity exhausting finite resources, classify as **#6 Flooding Attack**. If the primary mechanism is an implementation defect that causes crash/hang/degradation (including algorithmic complexity), classify as **#2** or **#3** per R-ROLE.

**Reference:** §2.2.4 (R-FLOOD)

## R-HUMAN (Human Manipulation Isolation)

Global mapping rule: If the attacker's advantage comes from psychological manipulation of a human, that manipulation step MUST be classified as **#9 Social Engineering**, and any subsequent technical steps MUST be classified separately.

**Reference:** §2.2.4 (R-HUMAN)

## **R-INTRA-7 (Classification Independence) (V2.1)**

Global mapping rule: Intra-system boundaries (`[[type]][@from->@to]`) never change cluster classification. They are observability annotations, not classification inputs. The cluster assigned to a step is determined solely by the generic vulnerability exploited (per R-ROLE, R-EXEC, R-ABUSE, etc.).

**Reference:** §2.2.4 (R-INTRA), §3.3.6 (Intra-System Boundary Operator)

## **R-INTRA-9 (Reserved Boundary Type) (V2.1)**

Global mapping rule: The `memory` boundary type for the Intra-System Boundary Operator is explicitly deferred and **MUST NOT** be used. Memory-level transitions are reserved for potential future specification. Tools and validators **SHOULD** reject `[[memory]][@from->@to]` as non-conformant.

**Reference:** §2.2.4 (R-INTRA), §3.3.6 (Intra-System Boundary Operator)

## **R-MITM (Position vs Action)**

Global mapping rule: The method of gaining a privileged communication-path position maps to another cluster. `#5 Man in the Middle` begins only once the attacker controls a point on the communication path and performs MitM actions.

**Reference:** §2.2.4 (R-MITM)

## **R-PHYSICAL (Physical Domain Isolation)**

Global mapping rule: If the attacker's advantage comes from unauthorized physical interaction or interference with hardware, facilities, media, or signals, that step **MUST** be classified as `#8 Physical Attack`, and subsequent technical steps **MUST** be classified separately.

**Reference:** §2.2.4 (R-PHYSICAL)

## **R-ROLE (Server vs Client Determination)**

Global mapping rule: If the vulnerable component accepts and handles inbound requests relative to the attacker, classify as `#2 Exploiting Server`. If the vulnerable component consumes external responses/content relative to the attacker, classify as `#3 Exploiting Client`.

**Reference:** §2.2.4 (R-ROLE)

## R-SUPPLY (Trust Acceptance Event Placement)

Global mapping rule: `#10 Supply Chain Attack` MUST be placed at the Trust Acceptance Event (TAE)—the moment where the third-party trust link is honored and the trust artifact becomes authoritative inside the organization's domain.

**Reference:** §2.2.4 (R-SUPPLY)

## R-TRANSIT-3 (Vendor Code on Target Device) (V2.1)

Global mapping rule: Vendor code running on the target device is NOT transit. It is the attack surface and MUST be classified by R-ROLE. For example, a browser (Safari, Chrome) on the victim's phone is a client-role component (`#3 Exploiting Client`), not a transit party. The transit operator ( $\Rightarrow$ ) is reserved for entities that relay or carry the attack between spheres without processing the exploit payload on the target's behalf.

**Reference:** §2.2.4 (R-TRANSIT), §3.3.5 (Transit Boundary Operator)

## Realtime Velocity Class (V2.0)

A velocity classification where attack progression occurs within seconds or milliseconds. Typical threat clusters: #6 (Flooding), #2 (Wormable Exploits). Control strategy must rely on architecture, hardening, and circuit breakers, as detection and response times are insufficient. Example: DDoS attacks, wormable exploits like EternalBlue, automated exploitation frameworks.

## Regulatory Trigger Point (V2.0)

The specific event type in a TLCTC event chain that activates a regulatory notification or compliance obligation. Different regulations have different trigger points within the same attack sequence:

- **Data-triggered regulations** (e.g., GDPR Art. 33): Activate at E2 (Data Risk Event involving PII) — no PII affected means no GDPR notification obligation
- **Incident-triggered regulations** (e.g., NIS2 Art. 23): Activate at E1 (Significant Incident / System Compromise) — regardless of whether PII is involved

Understanding regulatory trigger points enables CISOs to build precise IR playbooks mapping specific RS container actions to logical triggers rather than generic "reporting checklists". See also: Propagated PR, Event Chain Length.

## Responsibility Sphere

The organizational owner of a domain, denoted as `@Entity`. Examples: `@Org`, `@Vendor`, `@Facilities`, `@HR`, `@CloudProvider`, `@MSP`. Different spheres have different policies, teams, governance structures, and potentially different legal boundaries. Domain boundary definitions identify where responsibility and control shift during an attack, which is critical for incident response, forensics, and legal responsibility. Defined in `tlctc-`



`responsibility-spheres.json` and customizable per organization. Standard spheres include: Attacker Side, Third-Party/Vendor Side, Victim Side, Shared/Transit. Used in conjunction with the domain boundary operator (||) in attack path notation.

**Reference:** §2.2.2 (Global Definitions), §3.4, §5.1.2

## Risk Event

In the TLCTC Bow-Tie model, the central occurrence that represents the materialization of a threat, positioned between causes (threats) and effects (consequences). For cyber risk, this is "Loss of Control" or "System Compromise". Can trigger event chains where one outcome becomes the event triggering subsequent events.

## Role Determination

Classification of a component as server-role or client-role based on its behavior in the specific interaction being classified. The same software product MAY appear as server-role in one interaction and client-role in another. Classification MUST follow the role of the component being exploited in the step.

**Reference:** §2.2.2 (Global Definitions), R-ROLE (§2.2.4)

## RS Container (Respond Container) (V2.0)

The logical collection of RESPOND-function controls and actions for a specific event ( $E_n$ ) in the TLCTC event chain. An RS Container holds:

- **Direct Response Actions:** Containment, eradication, forensics for the event itself
- **Propagated PR Controls:** Protection requirements inherited from downstream compliance events (E3a, E3b, etc.)

Notation:  $RS(E_n) = \{ \text{Response} \} \cup \{ \text{Propagated PR}(E_{n+1}) \} \cup \{ \text{Propagated PR}(E_{n+x}) \}$ . Example: RS(E1) for a ransomware incident may contain both incident containment actions AND propagated GDPR/NIS2 notification controls, each with distinct timelines and reporting authorities. Aligns with NIST CSF RESPOND function. See also: Propagated PR, Regulatory Trigger Point.

## S

## Secure Software Development Life Cycle (SSDLC)

A structured approach to embedding security throughout the software development process. The TLCTC framework integrates into each SSDLC phase, with programmer-level decisions during Requirements and Design, coder-level implementation during the Implementation phase, and both roles contributing to verification during Testing and ongoing vigilance during Maintenance.

## Sequence

The ordered progression of threat clusters in an attack. The TLCTC framework recognizes that identified Top-Level Threats must also be seen as sequence components in the attack scenario, with attackers using these components in varying orders depending on their script (Axiom VIII). Example: A phishing attack might follow #9→#3→#7, or a more complex attack might be #9→#7→#4→(#1+#7).

### Sequence Operator (→)

The operator meaning: the right-hand step occurs after the left-hand step, and the left-hand step enables or makes possible the right-hand step in the described scenario. ASCII alternative: `->`.

**Reference:** §3.1 (Sequence Operator)

## Server-Role Component

A component that **accepts and handles inbound requests or stimuli** relative to the attacker. The component is in "server role" for the specific interaction being classified.

**Reference:** §2.2.2 (Global Definitions), R-ROLE (§2.2.4)

## Social Engineering (#9)

A threat cluster where an attacker psychologically manipulates individuals into performing actions counter to their or their organization's best interests, such as divulging confidential information, granting access, executing code, or bypassing security procedures. The generic vulnerability is human psychological factors: gullibility, trust, ignorance, fear, urgency, authority bias, curiosity, or general compromisability. Often serves as the initial vector enabling other threat clusters (e.g., #9→#4 for credential harvesting, #9→#7 for malware installation, #9→#1 for feature misconfiguration).

## STIX (Structured Threat Information Expression)

A standardized language for representing cyber threat information. TLCTC V2.0 proposes enhancement by introducing TLCTC clusters as a new STIX Domain Object, adding attack path sequence objects, and enhancing attack pattern objects with cluster mappings, enabling standardized high-level threat categorization and attack sequence representation in threat intelligence sharing.

## Strategic Layer (Human-First)

A naming convention for TLCTC clusters using the format `#X` where  $X \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ . Used for executive communication, risk registers, board reporting, strategic planning, and high-level attack path discussion.

**Reference:** §2.2.1 (Two-Layer Naming Convention)

## Strategic Management Layer

The stable top-level layer of TLCTC containing the 10 clusters and their generic vulnerabilities. Used for governance, control mapping, and comparable incident documentation. Focuses on risk management, policy-making, and program governance, including threat cluster categorization, generic vulnerability identification, risk appetite and tolerance definition, security program management, compliance and governance, and resource allocation. Corresponds to cluster identifiers `#1–#10` or `TLCTC-XX.00`. Uses strategic notation (`#X`) for executive communication.

**Reference:** Axiom VIII (§1.2), §2.2.1

## Sub-Threat

Specific, detailed attack techniques or methods that fall within a broader Top Level Cyber Threat Cluster. Sub-threats represent the operational-level detail beneath the strategic-level clusters. Example: Under `#2 Exploiting Server`, sub-threats include SQL Injection, Buffer Overflows, RCE via Deserialization, SSRF, and XXE Injection.

## Supply Chain Attack (#10)

A top-level threat cluster on the cause side of the bow-tie, where an attacker compromises systems by abusing the trust relationship within an organization's supply chain. The attacker targets vulnerabilities in third-party software components, hardware, services, or distribution/update mechanisms that are **trusted and integrated** into the organization's own environment or products. The generic vulnerability is the necessary reliance on, and implicit trust placed in, external suppliers, vendors, components, and their associated development or distribution processes.

**Supply Chain as "Bridge Not Bucket":** `#10` is a *bridge* threat cluster that marks the use of a trusted supply-chain channel as an attack vector to cross from one domain/trust boundary into another (e.g. `@Vendor` → `@Org`). It does *not* absorb the semantics of other clusters (`#1–#9`).

### Three Key Supply-Chain Vectors (#10.x):

- **#10.1 Update Vector:** Post-deployment delivery/update flow compromise
- **#10.2 Development Vector:** Pre-deployment build/dev pipeline, repositories, or package ecosystem compromise
- **#10.3 Hardware Supply Chain Vector:** Hardware component, firmware, or manufacturing/assembly compromise

**Control-Level Third-Party Dependencies (Not #10):** Dependencies on third parties for patch delivery, security updates, or managed services are treated as *governance/control dependencies* on the right side of the bow-tie. They are **not themselves** a `#10` Supply Chain Attack unless the trusted integration channel is directly abused as an attack vector.

## System Compromise

Alternative term for "Loss of Control" in the Cyber Bow-Tie model, representing the central cyber risk event where an attacker gains unauthorized control over a system through exploitation of one or more threat clusters.

## T

### Techniques (TTPs)

Specific methods, procedures, and tactics that attackers use to exploit vulnerabilities and achieve their objectives. In the MITRE ATT&CK framework, techniques represent the operational "how" of attacks—the concrete actions adversaries take (e.g., T1190 "Exploit Public-Facing Application", T1566 "Phishing"). Techniques often reference specific vulnerabilities (CVEs) they exploit and the platforms/systems they target.

**Relationship to TLCTC:** Each MITRE technique can be mapped to one or more TLCTC clusters based on the generic vulnerability being exploited:

- T1190 (Exploit Public-Facing Application) → #2 Exploiting Server
- T1566.001 (Spearphishing Attachment) → #9 Social Engineering → #3 or #7 (depending on payload)
- T1078 (Valid Accounts) → #4 Identity Theft

**Key distinction:** Techniques describe attacker actions and behaviors (operational detail), while TLCTC clusters categorize the fundamental vulnerabilities being exploited (strategic framework). TLCTC V2.0 proposes enhancing MITRE ATT&CK by adding cluster mappings and typical velocity attributes to each technique.

See also: TTP, Sub-Threat, MITRE ATT&CK, Operational Layer, Weakness

### Temporal Notation (V2.0)

The V2.0 extension to standard attack path notation that explicitly annotates time intervals between threat cluster transitions ( $\Delta t$ ) using the format  $\rightarrow[\text{time}]$ . Time units include seconds (s), minutes (m), hours (h), days, weeks, months. Examples:

- Basic:  $\#9 \rightarrow [24h] \#4 \rightarrow [12m] \#1$
- With domain boundaries:  $\#9 \rightarrow [\text{days}] \#4 \rightarrow [\text{mins}] \#1 \quad || \quad [\text{dev}]$   
 $[\text{@Vendor} \rightarrow \text{@Org}] \quad || \quad \rightarrow [\text{weeks}] \#10.2 \rightarrow [0s] \#7$
- With parallel execution:  $\#9 \rightarrow [30s] \#7 \rightarrow [2m] \#4 \rightarrow [15m] (\#1 + \#7)$

Enables precise velocity analysis, detection coverage score calculation, and realistic assessment of control effectiveness against time-sensitive attacks.

## Transit Boundary Operator ( $\Rightarrow$ ) (V2.1)

Notation: `[[[context][@Source $\Rightarrow$ @Carrier $\Rightarrow$ @Target]]]`. An extension to the Domain Boundary Operator that marks responsibility spheres which **carry or relay** the attack without being the source or the target. The  $\Rightarrow$  symbol denotes transit (relay), while  $\rightarrow$  denotes delivery to the final target. Chained transit uses right-to-left relay order: `[[[context][@Source $\Rightarrow$ @CarrierB $\Rightarrow$ @CarrierA $\Rightarrow$ @Target]]]`. Transit is distinct from **#10 Supply Chain Attack**: transit marks a passive relay, while **#10** marks a Trust Acceptance Event. Key rule (R-TRANSIT-3): vendor code running on the target device is NOT transit — it is the attack surface (classify by R-ROLE). Example: **#9** `[[human][@Attacker $\Rightarrow$ @SMSProvider $\Rightarrow$ @Victim]]]` — phishing SMS relayed through carrier.

**Reference:** §3.3.5 (Transit Boundary Operator)

## Third-Party Trust Link (TTL)

Any reliance relationship where a third party can influence your domain. Examples: software components/libraries/dependencies, update/distribution channels, federation relationships (IdP/SP), managed control planes, SaaS admin consoles, signing/attestation/provenance chains, firmware/hardware supply chains, CI/CD pipeline integrations.

**Reference:** §2.2.2 (Global Definitions), §2.1 (#10 Definition)

## Threat (in TLCTC)

An initiating force that exploits a generic vulnerability and can trigger the central event (Loss of Control), implemented as a set of tactics, techniques, and procedures (TTP) that attackers apply to provoke an event or incident. In TLCTC, threats are implemented as the 10 Top Level Cyber Threat Clusters, each defined by exactly one generic vulnerability. Threats are positioned on the cause side of the Bow-Tie model, distinct from vulnerabilities, events, and consequences (Axiom III). Threats are NOT outcomes, actors, or control failures.

**Reference:** §1.4.1 (Bow-Tie Structure), Axioms III–V (§1.2)

## Threat Cluster

An organizational construct that groups a set of threats exploiting a common generic vulnerability related to IT systems and humans. The TLCTC framework defines 10 mutually exclusive threat clusters, each associated with a unique generic vulnerability (Axiom I) and distinct attack vector (Axiom II).

## Threat Topology

A structural property of TLCTC describing whether a threat cluster (or a concrete attack step) operates primarily within the software domain's technical attack surfaces (**internal**) or enables crossing domain boundaries (**bridge**).

**Reference:** §5.0 (Topology in TLCTC), §5.1 (Definitions)

## Tie-Breaker Rules

Precedence rules applied when a step appears to fit multiple clusters. Applied in order: (1) classify by initial generic vulnerability, (2) implementation flaw vs legitimate function misuse, (3) credential use always wins for the use step, (4) MitM starts at controlled position, (5) flooding is about capacity, (6) FEC execution must be explicit, (7) human/physical/third-party are not shortcuts, (8) document non-obvious decisions.

**Reference:** §2.2.5 (Tie-Breaker / Precedence Rules)

## TLCTC (Top Level Cyber Threat Clusters)

A pragmatic and structured framework for targeted threat identification that provides a universal approach to cybersecurity applicable across diverse IT systems and contexts. The framework consists of 10 distinct, non-overlapping threat clusters based on generic vulnerabilities, each with strategic and operational applications. The 10 Top Level Cyber Threat Clusters are:

1. Abuse of Functions
2. Exploiting Server
3. Exploiting Client
4. Identity Theft
5. Man in the Middle (MitM)
6. Flooding Attack
7. Malware
8. Physical Attack
9. Social Engineering
10. Supply Chain Attack

## TLCTC Enumeration

A structured identifier system (`TLCTC-XX.YY`) where:

- `TLCTC-` prefix ensures proper attribution to the model
- `XX` represents the primary cluster number (01-10), zero-padded for consistent formatting
- `.YY` suffix designed for future refinement (`.00` designates current high-level definitions)

This provides machine readability, consistent sorting, and extensibility for sub-categorization.

## **Trust Acceptance Event (TAE)**

The moment your domain **honors** the Third-Party Trust Link and treats a Trust Artifact/Decision as authoritative. Actions at TAE include: validate, accept, install, apply, execute, attach privileges. **#10 Supply Chain Attack** is placed at the TAE.

**Reference:** §2.2.2 (Global Definitions), R-SUPPLY (§2.2.4), §2.1 (#10 Definition)

## **Trust Artifact / Trust Decision (TAD)**

What crosses the boundary and is accepted as authoritative in a third-party trust relationship. Examples: SAML/OIDC assertions, federated tokens, signed packages/updates/container images, CI build artifacts/release binaries, policy/configuration pushes, admin actions from managed platforms, firmware images.

**Reference:** §2.2.2 (Global Definitions), §2.1 (#10 Definition)

## **TTP (Tactics, Techniques, and Procedures)**

A detailed description of attacker behavior. In the TLCTC framework, specific TTPs (like those in MITRE ATT&CK) are considered instances or sub-threats that are categorized under the broader, cause-oriented Top Level Cyber Threat Clusters.

## **Two-Tiered Approach**

The TLCTC structure distinguishing between:

- **Strategic Management Layer:** High-level risk management, policy-making, and governance using the 10 Top Level Cyber Threat Clusters
- **Operational Layer:** Detailed implementation of controls, specific vulnerability management, and threat intelligence using sub-threats and TTPs

## **U**

## **Umbrella Controls**

Security measures that provide protection for groups of IT systems within their scope, such as firewalls, proxies, network zones, or external network filters. These contrast with Local Controls that protect specific systems directly. Important consideration: Umbrella controls provide protection only for specific 'Groups of IT-Systems' within their scope and cannot effectively protect all exposed systems (e.g., a firewall protects 'inner IT-Systems' but not directly exposed ones).

## Unknown $\Delta t$

A  $\Delta t$  value where no supported time statement can be made. Notation:  $\Delta t=?$ .

**Reference:** §4.0.3, §4.2.3

## V

### Velocity Annotation

Notation:  $\rightarrow[\Delta t=\text{value}]$  or  $\rightarrow[\Delta t=Xh]$ ,  $\rightarrow[\Delta t=Xm]$ ,  $\rightarrow[\Delta t=Xs]$ . Indicates the observed or estimated time interval between one Attack Step and the next. Velocity annotations are OPTIONAL but RECOMMENDED for operational analysis and threat intelligence sharing.

**Reference:** §2.2.2 (Global Definitions), §3.5.2, §4.2 ( $\Delta t$  Notation)

### Velocity Class

Categorical labels for  $\Delta t$  ranges that describe the defender's feasible response mode and determine appropriate control strategies. Four classes are defined:

- **VC-1: Strategic / Latent/Slow** (days → months): Log retention, threat hunting, strategic monitoring
- **VC-2: Tactical / Medium** (hours): SIEM alerting, analyst triage, guided response
- **VC-3: Operational / Fast** (minutes): Automation (SOAR/EDR), rapid containment, prebuilt playbooks
- **VC-4: Real-Time** (seconds → milliseconds): Architecture & circuit breakers, rate-limits, hardening, automatic isolation

**Reference:** §4.4 (Operational Velocity Classes)

### Vertical Stack Application

The implementation of TLCTC across the layered architecture of IT systems (from application level to hardware), analyzing client/server roles at each protection ring boundary (e.g., Ring 3 to Ring 0) and directional vulnerabilities.

### Vulnerability

An exploitable condition or weakness in a system that can be leveraged by a threat actor to compromise security. In the TLCTC framework, vulnerabilities exist in a hierarchical relationship: specific vulnerabilities (like individual CVEs) are instances of generic vulnerabilities, which in turn define the 10 Top Level Cyber Threat Clusters (Axiom I).

**Conceptual hierarchy:** Weakness (CWE) → Specific Vulnerability (CVE) → Generic Vulnerability (TLCTC) → Threat Cluster (#1-#10).



**Critical distinction:** A vulnerability is an exploitable condition that exists in a system, while a weakness is the underlying flaw, bug, or error that enables that vulnerability to exist. TLCTC focuses on categorizing the generic vulnerabilities that all specific vulnerabilities map to.

See also: Generic Vulnerability, Weakness, CVE, Threat Cluster

## W

### Weakness

A flaw, bug, or error in software, hardware, or processes that enables vulnerabilities to exist. In the Common Weakness Enumeration (CWE) framework, weaknesses are categorized as the root causes of software security problems (e.g., CWE-89 for SQL Injection weakness, CWE-119 for buffer overflow weakness).

**Critical distinction in TLCTC context:** CWE categorizes weaknesses (the flaws themselves), not vulnerabilities (the exploitable conditions those flaws create). In the TLCTC framework, the conceptual hierarchy flows: **Weakness** → **Specific Vulnerability (CVE)** → **Generic Vulnerability** → **Threat Cluster**.

**Example:** A coding error that fails to validate input (weakness) creates a SQL injection vulnerability (specific vulnerability), which exploits the generic vulnerability of "server-side code flaws" (#2 Exploiting Server). TLCTC's 10 generic vulnerabilities represent the universal categories that all specific vulnerabilities ultimately map to, regardless of their underlying weaknesses.

**Relationship to TLCTC:** While CWE provides granular weakness taxonomy at the code level for developers, TLCTC operates at the strategic level by grouping all resulting vulnerabilities into 10 generic vulnerability categories that define the threat clusters. Both frameworks are complementary.

See also: Vulnerability, Generic Vulnerability, CVE, CWE, Threat Cluster, Coder, Programmer

## Quick Reference Tables

### Cluster Quick Reference

| #  | Name               | Generic Vulnerability                                  | Topology |
|----|--------------------|--------------------------------------------------------|----------|
| #1 | Abuse of Functions | Functional scope/trust (designed capabilities abused)  | Internal |
| #2 | Exploiting Server  | Server-side code implementation flaws                  | Internal |
| #3 | Exploiting Client  | Client-side code implementation flaws                  | Internal |
| #4 | Identity Theft     | Identity-artifact binding / credential lifecycle (use) | Internal |
| #5 | Man in the Middle  | Lack of end-to-end communication protection            | Internal |
| #6 | Flooding Attack    | Finite capacity limitations                            | Internal |

| #   | Name                | Generic Vulnerability                               | Topology |
|-----|---------------------|-----------------------------------------------------|----------|
| #7  | Malware             | Designed execution capability for untrusted content | Internal |
| #8  | Physical Attack     | Physical accessibility/interference                 | Bridge   |
| #9  | Social Engineering  | Human psychological factors                         | Bridge   |
| #10 | Supply Chain Attack | Third-party trust dependencies                      | Bridge   |

**Reference:** §2.1 (Cluster Definitions), §5.2 (Topology Classification)

### Axiom Quick Reference

| #           | Name                           | Group          | Core Statement                                               |
|-------------|--------------------------------|----------------|--------------------------------------------------------------|
| <b>I</b>    | No System-Type Differentiation | Scope          | Generic IT assets; sector labels don't create threat classes |
| <b>II</b>   | Client–Server Model            | Scope          | Universal interaction abstraction                            |
| <b>III</b>  | Causes, Not Outcomes           | Separation     | Threats $\neq$ data risk events                              |
| <b>IV</b>   | Not Threat Actors              | Separation     | Threats $\neq$ actor identity                                |
| <b>V</b>    | Not Control Failure            | Separation     | Control-risk $\neq$ threat category                          |
| <b>VI</b>   | Single-Cluster Rule            | Classification | One step = one vulnerability = one cluster                   |
| <b>VII</b>  | Initial-Vulnerability Rule     | Classification | Vector defined by initial generic vulnerability              |
| <b>VIII</b> | Strategic–Operational Layering | Classification | Clusters $\rightarrow$ sub-threats                           |
| <b>IX</b>   | Sequence + Velocity            | Sequences      | Clusters chain; $\Delta t$ measures velocity                 |
| <b>X</b>    | Credential Duality             | Sequences      | Acquisition vs application                                   |

**Reference:** §1.2 (Axioms and Assumptions)

### R-\* Rules Quick Reference

| Rule            | Distinguishes         | Key Decision                                                                                     |
|-----------------|-----------------------|--------------------------------------------------------------------------------------------------|
| <b>R-ROLE</b>   | #2 vs #3              | Server-role (accepts inbound) $\rightarrow$ #2; Client-role (consumes external) $\rightarrow$ #3 |
| <b>R-CRED</b>   | Acquisition vs Use    | Acquisition $\rightarrow$ enabling cluster; Use $\rightarrow$ always #4                          |
| <b>R-MITM</b>   | Gaining vs Exploiting | Gaining position $\rightarrow$ enabling cluster; Exploiting position $\rightarrow$ #5            |
| <b>R-FLOOD</b>  | Capacity vs Defect    | Volume exhaustion $\rightarrow$ #6; Implementation defect $\rightarrow$ #2/#3                    |
| <b>R-EXEC</b>   | FEC Execution         | If FEC executes $\rightarrow$ #7 MUST be recorded (plus enabling cluster)                        |
| <b>R-SUPPLY</b> | TAE Placement         | #10 at Trust Acceptance Event where third-party trust is honored                                 |

| Rule                         | Distinguishes               | Key Decision                                                          |
|------------------------------|-----------------------------|-----------------------------------------------------------------------|
| <b>R-HUMAN</b>               | Human Manipulation          | Psychological manipulation → #9; subsequent tech steps separate       |
| <b>R-PHYSICAL</b>            | Physical Access             | Physical interaction → #8; subsequent tech steps separate             |
| <b>R-ABUSE</b>               | Function Misuse             | No flaw required, legitimate capability abused → #1                   |
| <b>R-TRANSIT-3</b><br>(V2.1) | Transit vs Attack Surface   | Vendor code on target device → classify by R-ROLE, not transit        |
| <b>R-INTRA-7</b><br>(V2.1)   | Intra-System Classification | Intra-system boundaries are observability only → never change cluster |
| <b>R-INTRA-9</b><br>(V2.1)   | Reserved Boundary Type      | memory boundary type is deferred → MUST NOT be used                   |

**Reference:** §2.2.4 (Global Mapping Rules), §2.2.9 (Quick Reference)